

Giới thiệu dự án gốc

Ứng dụng [forum](#) xây dựng bằng ngôn ngữ Golang, cho phép người dùng tương tác với nhau thông qua các bài đăng, bình luận và reaction.

Tính năng chính của dự án gốc

1. Authentication & Session Management

- Cơ chế: Cookie-based Authentication.
- Bảo mật: Password hashing.
- Session: Kiểm tra tính hợp lệ của session trên mỗi request (ValidSession()).

2. Tương tác người dùng

- Bài viết: Đăng bài viết theo các danh mục.
- Tương tác: Bình luận và thả cảm xúc.
- Spam protection: Giới hạn tốc độ comment bằng throttling.

3. Filtering & Pagination

- Theo danh mục: /category/{id}
- Bài viết đã tạo: /mycreatedposts
- Bài viết đã thích: /mylikedposts
- Phân trang: Tối ưu hóa query với tham số PageID.

Các cải tiến

1. Tối ưu hóa Database

1.1. Hệ thống gốc sử dụng SQLite làm database:

- Vấn đề:
 - + SQLite khóa cả database khi có thao tác chèn/ xóa/ sửa, chỉ cho một tiến trình ghi ở 1 thời điểm.
 - + Database là 1 file lưu ở ổ cứng không phải server.
- Các hướng giải quyết:
 - + Sử dụng MySQL:
Chỉ khóa hàng khi có thao tác chèn/ xóa/ sửa, nhiều tiến trình ghi được.

Database là server riêng, có khả năng scale được.

Biểu diễn được các quan hệ khóa ngoại như ở database gốc (vd: user_id là khóa ngoại của post_id), dùng được on update cascade, on delete cascade, ... (VD: xóa user thì xóa cả post)

- + Sử dụng MongoDB:

Khóa ở mức document, database cũng là server riêng.

Tuy nhiên không biểu diễn được quan hệ khóa ngoại, các câu lệnh on update cascade, on delete cascade, lập trình viên phải tự cài đặt logic đó (để lỗi, bỏ sót, ...).

-> Vì vậy nhóm em chọn sử dụng MySQL.

1.2. Subquery

- Vấn đề: Ở mã nguồn ban đầu, việc truy vấn các post thực hiện bằng cách: truy vấn ở bảng post để tìm các post, ứng với một post lại thực hiện truy vấn ở các bảng comment, like, dislike, category => phải thực hiện nhiều query, đặc biệt là ở trang dashboard (load 10 post, mỗi post thực hiện 4 query nữa => tổng 40 + 1 query)

- Hướng giải quyết: Nhóm đã thêm bảng là materialized_view của post, ở đây đã tính toán sẵn số comment, số like, số dislike và category. Vì vậy khi lấy thông tin tổng quan của post, chỉ cần làm 1 query duy nhất là lấy được các thông tin tổng quan của post.

- Đánh đổi: Khi cài đặt materialized view thì có thể thời gian ghi sẽ lâu hơn (do phải cập nhật thêm bảng, tính toán các giá trị comment, like, ... để cập nhật ở bảng materialized view). Tuy nhiên với đặc thù của hệ thống forum là đọc nhiều hơn ghi thì nhóm thấy thời gian ghi chậm hơn một chút đáng để đánh đổi cho thời gian đọc, ngoài ra kết quả kiểm thử cho thấy thời gian ghi tương đối nhanh.

1.3. Indexing

- Vấn đề: Ở mã nguồn ban đầu, với các bảng như posts, sessions, khi thực hiện sắp xếp bài viết theo thời gian (ORDER BY created_at) để hiển thị lên dashboard, hay tìm kiếm bài viết theo người dùng (WHERE user_id = ...), CSDL sẽ phải thực hiện quét toàn bộ bảng, làm tăng thời gian đọc khi số lượng bản ghi ở CSDL tăng lên.

- Hướng giải quyết: Nhóm đã áp dụng Indexing bằng cách thêm INDEX một cách tường minh ở schema: Ví dụ:

- + Thêm INDEX idx_created_at (created_at) vào bảng posts và post_materialized_view để sắp xếp bài viết theo thời gian
- + Thêm INDEX idx_user_id (user_id) vào bảng posts để truy xuất nhanh danh sách bài viết của một user cụ thể.

- Đánh đổi: Thêm index làm tăng không gian để lưu trữ bảng index. Ngoài ra, thời gian ghi có thể lâu hơn vì CSDL phải cập nhật lại các bảng index nếu CSDL có thay đổi (thêm/ xóa/ sửa/).

Kết quả kiểm thử thu được như sau:

Kịch bản	Chỉ số	Trước cải tiến	Sau cải tiến
Tổng thể	Tổng req	10,628	8,472
	Req rate	49.34	16.2
	Failed req rate	22.59%	0.00%
Read Only	Response Time P95	642.91ms	15,642ms
	Response Time Avg	307.47ms	9,941ms
	Response Time Max	1,464ms	18,746ms
	Success rate	99.63%	100.00%
Write Only	Response Time P95	-	248ms
	Response Time Avg	-	1,069ms
	Response Time Max	-	1,638ms
	Success rate	0%	100.00%
Mix (50% read, 50% write)	Response Time P95	5,023ms	5,543ms
	Response Time Avg	1,434ms	898ms
	Response Time Max	10,658ms	11,222ms
	Create Posts success rate	0%	100%

2. Caching layer

Sau khi đã chuyển đổi Database từ SQLite sang MySQL, vấn đề về khả năng ghi của ứng dụng đã được giải quyết khi đã không còn error nhưng khả năng query đến các endpoint /category/<>?PageID=<> vẫn rất lâu, vì vậy nhóm em sẽ thêm caching layer. Layer này sẽ cache toàn bộ post và category của nó để sau tránh phải query lấy category. Đồng thời nó cũng cache riêng lẻ nội dung từng post. Sau khi thêm caching layer này, kết quả kiểm thử

Kịch bản	Chỉ số	Trước cải tiến	Sau cải tiến
Tổng thể	Tổng req	8,472	12,655
	Req rate	16.2	59.15
	Failed req rate	0.00%	0.00%
Read Only	Response Time P95	15,642ms	8.84ms
	Response Time Avg	9,941ms	7.26ms
	Response Time Max	18,746ms	233.64ms
	Success rate	100.00%	100.00%
Write Only	Response Time P95	248ms	647.18ms
	Response Time Avg	1,069ms	161.89ms
	Response Time Max	1,638ms	929.90ms
	Success rate	100.00%	100.00%
Mix (50% read, 50% write)	Response Time P95	5,543ms	82.74ms
	Response Time Avg	898ms	40.95ms
	Response Time Max	11,222ms	658.21ms
	Create Posts success rate	100%	100.00%

3. Rate limiting

- Vấn đề:

Với hệ thống forum gốc, tác giả đã sử dụng throttling trên frontend để ngăn người dùng spam request, tuy nhiên như thế là không đủ, hệ thống vẫn có thể bị DDos, lạm dụng API, database gây ảnh hưởng đến hiệu năng của hệ thống

- Giải quyết:

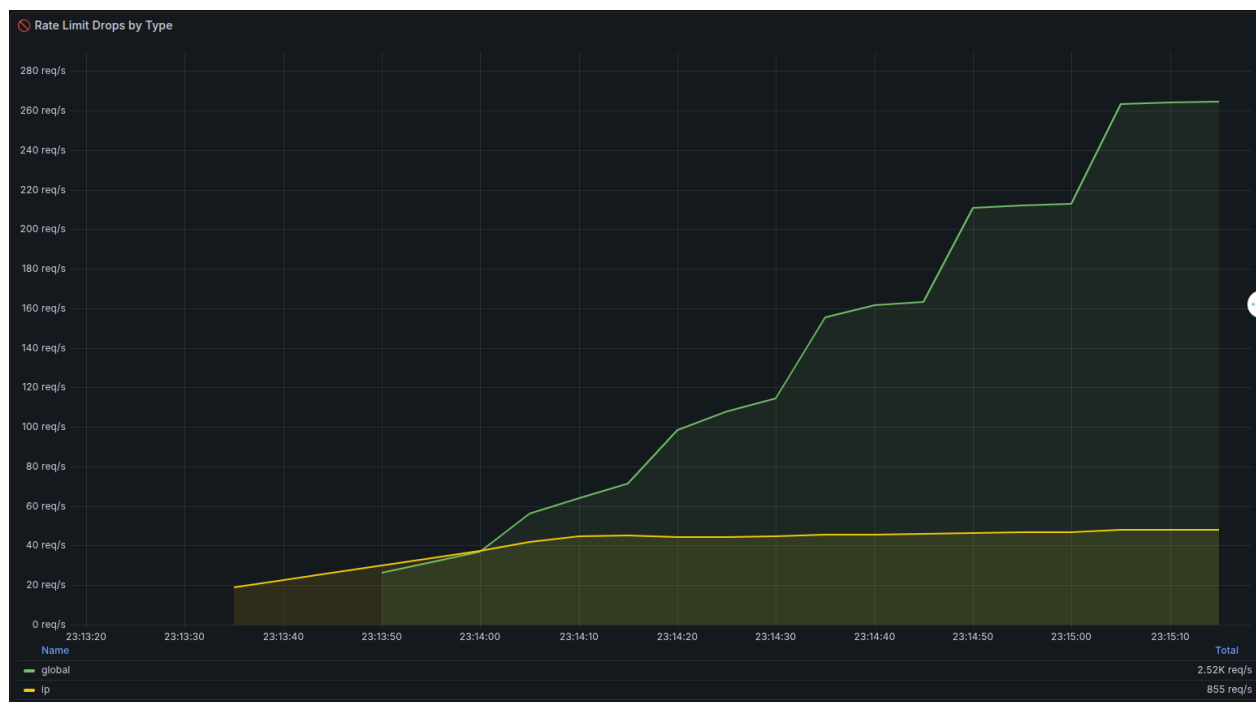
Sử dụng rate limiting: Global, Per User, Per IP, Per endpoint. Từ đó có thể chặn các request hàng loạt, đồng thời, bảo vệ được CPU, Database, ngăn các request spam và làm ổn định hệ thống, từ chối request để hệ thống không bị quá tải.

- Đánh đổi: khi dùng rate limiting thì cũng phải đánh đổi một số thứ, trong đó, trải nghiệm người dùng có thể bị giảm, các user hợp lệ đôi lúc bị block hoặc chậm khi quá ngưỡng cho phép. Nhưng đây là sự đánh đổi cần thiết để hệ thống có thể chịu được các tải bất thường, bảo vệ được các layer phía sau.
- Kết quả: Sau khi áp dụng ratelimit, chạy test k6 cho 1 user spam request đến hệ thống, hệ thống sẽ chặn các request spam theo user IP đó

```

2025/12/05 16:14:05 Error scanning row: Maximum retry attempts reached: sql: no rows in result set
{"level":"error","service":"forum","method":"GET","path":"/post/460","ip":"172.18.0.1","user_id":-1,"error":"Maximum retry attempts reached: sql: no rows in result set","post_id":460,"status":500,"duration_ms":2404.950303,"time":"2025-12-05T16:14:05Z","message":"Failed to fetch post"}
2025/12/05 16:14:05 RATE LIMIT EXCEEDED | IP: 172.18.0.1 | Path: / | Method: GET
{"level":"info","service":"forum","method":"GET","path":"/","ip":"172.18.0.1","user_id":-1,"time":"2025-12-05T16:14:05Z","message":"Fetching posts"}
{"level":"info","service":"forum","method":"GET","path":"/","ip":"172.18.0.1","user_id":-1,"cache_key":"index_posts_page_0","cache_hit":true,"post_count":10,"duration_ms":0.011746,"time":"2025-12-05T16:14:05Z","message":"Posts fetched from cache"}
2025/12/05 16:14:05 RATE LIMIT EXCEEDED | IP: 172.18.0.1 | Path: /post/476 | Method: GET
2025/12/05 16:14:05 RATE LIMIT EXCEEDED | IP: 172.18.0.1 | Path: / | Method: GET
2025/12/05 16:14:05 RATE LIMIT EXCEEDED | IP: 172.18.0.1 | Path: /signup | Method: POST
2025/12/05 16:14:05 RATE LIMIT EXCEEDED | IP: 172.18.0.1 | Path: / | Method: GET
2025/12/05 16:14:05 RATE LIMIT EXCEEDED | IP: 172.18.0.1 | Path: /mylikedposts | Method: GET
2025/12/05 16:14:05 RATE LIMIT EXCEEDED | IP: 172.18.0.1 | Path: /mylikedposts | Method: GET
2025/12/05 16:14:05 RATE LIMIT EXCEEDED | IP: 172.18.0.1 | Path: / | Method: GET
2025/12/05 16:14:05 RATE LIMIT EXCEEDED | IP: 172.18.0.1 | Path: / | Method: GET
2025/12/05 16:14:05 RATE LIMIT EXCEEDED | IP: 172.18.0.1 | Path: /signin | Method: POST
2025/12/05 16:14:05 RATE LIMIT EXCEEDED | IP: 172.18.0.1 | Path: / | Method: GET
2025/12/05 16:14:05 RATE LIMIT EXCEEDED | IP: 172.18.0.1 | Path: /category/1 | Method: GET
{"level":"info","service":"forum","method":"GET","path":"/","ip":"172.18.0.1","user_id":-1,"time":"2025-12-05T16:14:05Z","message":"Fetching posts"}
{"level":"info","service":"forum","method":"GET","path":"/","ip":"172.18.0.1","user_id":-1,"cache_key":"index_posts_page_0","cache_hit":true,"post_count":10,"duration_ms":0.014926,"time":"2025-12-05T16:14:05Z","message":"Posts fetched from cache"}
2025/12/05 16:14:05 RATE LIMIT EXCEEDED | IP: 172.18.0.1 | Path: / | Method: GET
2025/12/05 16:14:05 RATE LIMIT EXCEEDED | IP: 172.18.0.1 | Path: /signin | Method: POST
2025/12/05 16:14:05 RATE LIMIT EXCEEDED | IP: 172.18.0.1 | Path: / | Method: GET
2025/12/05 16:14:05 RATE LIMIT EXCEEDED | IP: 172.18.0.1 | Path: / | Method: GET
2025/12/05 16:14:05 Max attempts reached. Last error:sql: no rows in result set
{"level":"error","service":"forum","method":"GET","path":"/post/335","ip":"172.18.0.1","user_id":-1,"error":"Maximum retry attempts reached: sql: no rows in result set","post_id":335,"status":500,"duration_ms":2405.427158,"time":"2025-12-05T16:14:05Z","message":"Failed to fetch post"}
2025/12/05 16:14:05 Error scanning row: Maximum retry attempts reached: sql: no rows in result set
2025/12/05 16:14:05 RATE LIMIT EXCEEDED | IP: 172.18.0.1 | Path: /post/293 | Method: GET
2025/12/05 16:14:05 RATE LIMIT EXCEEDED | IP: 172.18.0.1 | Path: /post/253 | Method: GET
2025/12/05 16:14:05 RATE LIMIT EXCEEDED | IP: 172.18.0.1 | Path: /mycreatedposts | Method: GET
2025/12/05 16:14:05 RATE LIMIT EXCEEDED | IP: 172.18.0.1 | Path: / | Method: GET
2025/12/05 16:14:05 RATE LIMIT EXCEEDED | IP: 172.18.0.1 | Path: /post/366 | Method: GET
2025/12/05 16:14:05 RATE LIMIT EXCEEDED | IP: 172.18.0.1 | Path: / | Method: GET
2025/12/05 16:14:05 RATE LIMIT EXCEEDED | IP: 172.18.0.1 | Path: /post/104 | Method: GET

```



4. Reliability & Monitoring

4.1. Retry pattern

a) Về phía backend

Để tránh các trường hợp database bị khởi tạo sau app, hoặc database chưa sẵn sàng vào thời điểm tạo connection, query, v.v. Nhóm em đã thực hiện retry đối với các thao tác khởi tạo kết nối tới DB hay query tới DB với các config khác nhau.

Nhóm em đã giả lập một ví dụ thực tế bằng cách tạo một hàm test, khi hàm đó truy cập vào database và lock toàn bộ bảng lại trong khi hàm đang chạy. Hàm này cũng được viết trên 2 logic khác nhau: có bọc trong hàm Retry và không bọc trong hàm Retry. Nhóm thực hiện 2 kịch bản sau:

- + Thực hiện query đơn lẻ
- + Nhiều user query song song bằng hàm test

Với kịch bản thực hiện query đơn lẻ thì kết quả chạy test kịch bản K6 đã pass ở cả hai logic. Tuy nhiên với kịch bản thứ 2, nếu không áp dụng hàm Retry, các request của user đến sau sẽ bị thất bại; khi áp dụng hàm Retry, các request của user sẽ được phản hồi dần dần qua các lần thử như hình chụp log ở dưới đây:

INFO[0023] [VU1] Query succeeded after 30 attempts	source=console
INFO[0023] [VU5] Query succeeded after 30 attempts	source=console
INFO[0023] [VU2] Query succeeded after 30 attempts	source=console
INFO[0023] [VU3] Query succeeded after 30 attempts	source=console
INFO[0023] [VU6] Query succeeded after 30 attempts	source=console
INFO[0025] [VU6] Query succeeded after 31 attempts	source=console
INFO[0025] [VU5] Query succeeded after 32 attempts	source=console
INFO[0026] [VU2] Query succeeded after 36 attempts	source=console
INFO[0026] [VU1] Query succeeded after 36 attempts	source=console

b) Về phía frontend

Để tránh người dùng mất nội dung post, comment được biên soạn nhưng không gửi được do lỗi mạng. Mỗi khi có sự kiện mất mạng, mình sẽ lưu post/comment hiện tại vào localStorage của trình duyệt. Trình duyệt có thể được tắt, hoặc bật tới khi có sự kiện có mạng trở lại, khi đó sẽ thực hiện lấy từ localStorage item có tên offline_queue do ứng dụng đã lưu trước đó, nếu có phần tử trong queue này thì thực hiện post lại theo nội dung được lưu.

4.2 Monitoring: Theo dõi các metrics sử dụng Prometheus + Grafana

- Vấn đề:

- + Hệ thống ban đầu chỉ dùng print ra các trạng thái, lỗi đơn giản, trong quá trình sử dụng, nếu xảy ra lỗi hoặc sự cố thì rất khó khăn trong việc kiểm tra, debug và khó bảo trì.

Giải quyết:

- + Sử dụng Prometheus để scrape các metric từ app như Uptime, System Health, HTTP Status, Response Time Percentiles (p95, p99), DB Latency, ..., sau đây sử dụng Grafana để visualize qua dashboard, do đó có thể theo dõi trạng thái của hệ thống, khi hệ thống gặp sự cố thì dễ dàng phát hiện lỗi ở đâu, giúp quá trình sửa lỗi và bảo trì nhanh hơn



5. Secure File Upload Architecture

Forum ban đầu chưa có tính năng upload ảnh cho bài viết. Chính vì vậy, bọn em đã thêm tính năng upload ảnh cho bài viết. Đi kèm với đó, có những vấn đề mới phát sinh trong việc xử lý và quản lý file ảnh được upload cùng bài viết

a) Gatekeeper Pattern

- Vấn đề: Cần phải kiểm soát những người dùng nào tạo post và upload ảnh. Ngoài ra, Server cần phải kiểm soát dung lượng, định dạng của file upload tránh việc spam file (kết hợp với Rate Limit ở Middleware) lên Cloud gây thiệt hại lớn về chi phí lưu trữ.
- Giải pháp: Tạo một chốt chặn (/api/upload/request-url) để kiểm tra metadata (Size, Type, Auth) trước khi cấp SAS Token cho phía Client để có thể upload file.
- Kiểm thử: Sử dụng cURL request POST đến /api/upload/request-url

- + Nếu gửi request với size > 5MB thì response trả về lỗi 400 (Bad Request error)
- + Nếu gửi request không kèm Cookie đăng nhập (session_token) thì response trả về lỗi 401 (Unauthorized error)

- Đánh đổi:

- + Ưu điểm: Hạn chế việc SAS Token có thể xin một cách bừa bãi để upload file. Từ đó, chi phí lưu trữ và xử lý ở trên Cloud sẽ được tiết kiệm khi hạn chế được việc spam file lên Cloud.
- + Nhược điểm: Tăng độ trễ nhẹ do phải thêm 1 round-trip (Client phải hỏi Server trước rồi mới upload được)

b) Valet Key Pattern

- Vấn đề: Nếu file ảnh từ phía client gửi lên Backend Server xử lý và xác thực rồi mới upload lên Cloud thì điều này làm cho Backend Server có thể bị quá tải vì phải xử lý cả chiều nhận và chiều gửi (tăng chịu tải gấp đôi). Từ đó, băng thông Server có thể bị nghẽn.

- Giải pháp: Server cấp cho Client một SAS Token có level là Blob-level SAS (chỉ dành cho 1 file cụ thể) có quyền ghi giới hạn. Client dùng SAS Token để upload thẳng lên Azure.

- Kiểm thử: Sử dụng k6 với 10 Virtual Users (VUs) upload file 1MB (vì hạn chế về mặt băng thông đường truyền mạng và chi phí xử lý lưu trữ Cloud)

Tiêu chí	Valet Key	Proxy Upload (Upload qua Server)
Tiêu chí	Valet Key	Proxy Upload (Upload qua Server)
Latency P50 (Median)	85.99 ms	2.28 s
Latency P95	3.13 s	4.61 s
Throughput:	5.899191 req/s	1.988268 req/s
Errors:	0/10	0/10

- Đánh đổi:

- + Ưu điểm: Hạn chế việc Server bị nghẽn, tận dụng được băng thông khổng lồ của Cloud. Scalability cao.

- + Nhược điểm: Cần quản lý Token cực kỹ (thời hạn ngắn, quyền tối thiểu) để tránh lộ lọt.

c) Quarantine Pattern

- Vấn đề: Hacker có thể dùng file sạch để xin Token, nhưng sau đó upload file virus.
- Giải pháp: Mọi file mới upload đều bị nhốt vào quarantine-container. Một Watcher sẽ kiểm tra file trong đó (lấy 512B đầu tiên của file để kiểm tra MagicByte và kiểm tra ContentLength). Chỉ khi an toàn mới được chuyển sang public-container
- Kiểm thử: Dùng cURL request POST đến /api/upload/request-url với file ảnh bình thường. Sau đó, lấy upload-url mà Server trả về rồi cURL PUT lên url đó một file khác (nặng hơn 5MB hoặc file định dạng khác ảnh). Những file đó sẽ bị Quarantine Watcher “bắt” và xóa chúng.
- Đánh đổi:
 - + Ưu điểm: Ngăn chặn hoàn toàn những file không phải định dạng ảnh.
 - + Nhược điểm: Phải chờ Watcher quét